Assignment

1.1 Design Code of Sequence Detector

```
 module sd (
input wire clk,
input wire rst,
input wire d,
output reg [3:0] out
);
parameter S_res  = 3'b000;
parameter S_0    = 3'b001;
parameter S_01   = 3'b010;
parameter S_010  = 3'b011;
parameter S_0101 = 3'b100;

reg [2:0] curr, next;
always @(posedge clk) begin
  if (!rst)
    curr <= S_res;
  else
    curr <= next;
end
always @(*) begin
  case (curr)
    S_res:  next = d ? S_res : S_0;
    S_0:    next = d ? S_01 : S_0;
    S_01:   next = d ? S_res : S_010;
    S_010:  next = d ? S_0101 : S_0;
    S_0101: next = d ? S_res : S_010;
    default: next = S_res;
  endcase
end
```

```
    always @(*) begin
        if (curr == S_0101)
            out = 4'b0100;   //Vowels=4
        else
end
endmodule
```

1.2 Testbench Code

```
`timescale 1ns/1ps
module tb_sd();

    reg clk, rst, d;
    wire [3:0] out;

    sd uut(clk,rst,d,out );

    always #5 clk = ~clk;

    initial begin
        $dumpfile("sd.vcd");
        $dumpvars(0, tb_sd);

        $monitor("time=%0t | rst=%b | d=%b | out=%0d (%b)",
                 $time, rst, d, out, out);

        clk = 0;
        rst = 0;
        d = 0;
        @(negedge clk); rst = 0;

        // Send 0101
        @(negedge clk); d = 0;
        @(negedge clk); d = 1;
        @(negedge clk); d = 0;
```

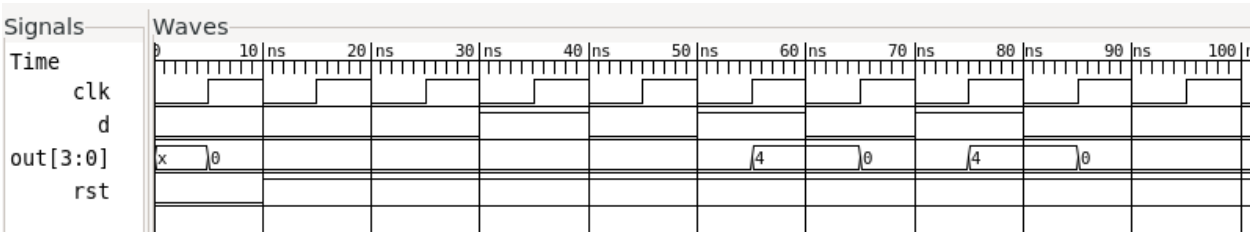
```
    @(negedge clk); d = 1;


    //overlapping detection
    @(negedge clk); d = 0;
    @(negedge clk); d = 1;
    @(negedge clk); d = 0;
    @(negedge clk); d = 0;
    @(negedge clk); d = 0;
    #1;
$finish;
  end
endmodule
```

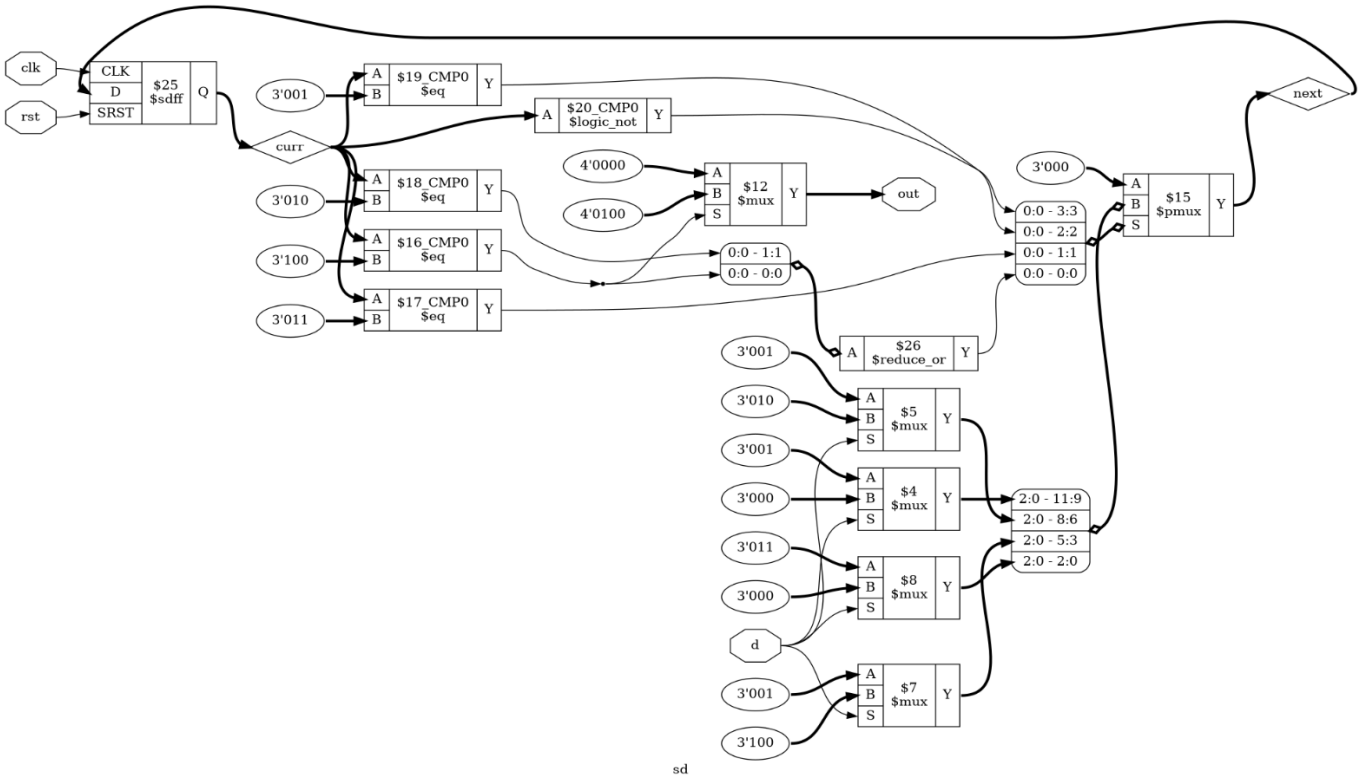
1.3 Terminal based Output reflecting the username through iverilog.

```
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ nano sd.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ nano tb_sd.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ iverilog -o sd_sim sd.v tb_sd.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ vvp sd_sim
VCD info: dumpfile sd.vcd opened for output.
time=0 | rst=0 | d=0 | out=x (xxxx)
time=5000 | rst=0 | d=0 | out=0 (0000)
time=10000 | rst=1 | d=0 | out=0 (0000)
time=30000 | rst=1 | d=1 | out=0 (0000)
time=40000 | rst=1 | d=0 | out=0 (0000)
time=50000 | rst=1 | d=1 | out=0 (0000)
time=55000 | rst=1 | d=1 | out=4 (0100)
time=60000 | rst=1 | d=0 | out=4 (0100)
time=65000 | rst=1 | d=0 | out=0 (0000)
time=70000 | rst=1 | d=1 | out=0 (0000)
time=75000 | rst=1 | d=1 | out=4 (0100)
time=80000 | rst=1 | d=0 | out=4 (0100)
time=85000 | rst=1 | d=0 | out=0 (0000)
tb_sd.v:38: $finish called at 101000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ gtkwave sd.vcd
```

1.4 Snapshot of waveforms viewed in GTKWave.



1.5 Schematic of the Final Synthesis through YOSYS.



2.1 Design of Mac Unit

```
module mac(
  input wire clk,rst,enable,
  input wire [7:0]in_A,
  input wire [3:0]V,
   output reg [7:0] acc
);


always@(posedge clk)begin
  if(!rst)begin
     acc<= 8'b0;
  end else begin
     if(enable)begin
       acc<= acc+(in_A*V);  //accumulator
     end else begin
       acc<=acc;   //hold
     end
  end
end
endmodule
```

2.2 Testbench Code

```
`timescale 1ns/1ps

module tb_mac();
parameter R = 65; // roll number
localparam [3:0] V = 4'd4; // vowel count

reg clk,rst,enable;
reg [7:0]in_A;
wire [7:0]acc;

mac uut(clk,rst,enable,in_A,V,acc);

always #5 clk=~clk;

initial begin
//initialization
clk=0;rst=0;enable=0;in_A=8'b0;

$dumpfile("mac.vcd");
```

```
$dumpvars(0, tb_mac);

$monitor("Time=%0t | rst=%b enable=%b | in_A=%d | acc=%d",
$time, rst, enable, in_A ,acc);

#R; rst=1;
//hold for exactly(R=65ns)

// Cycle--1
@(negedge clk);enable = 1;in_A=8'd2;

// Cycle--2
@(negedge clk);enable = 1;in_A = 8'd5;

// Cycle--3
@(negedge clk);enable = 0;in_A = 8'd10;

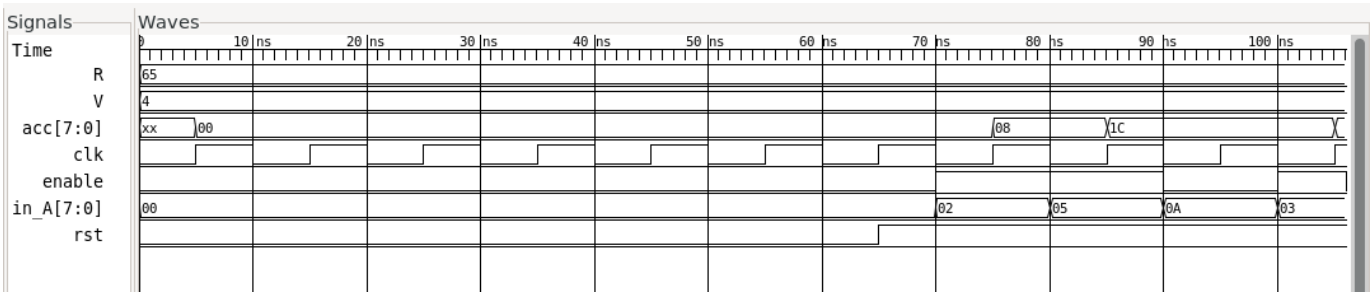

// Cycle--4
@(negedge clk);enable = 1;in_A = 8'd3;

// Allow MAC to process Cycle 4
@(posedge clk);
#1;
$display("Final accumulator = %0d", acc);
enable = 0;
$finish;
end
endmodule
```

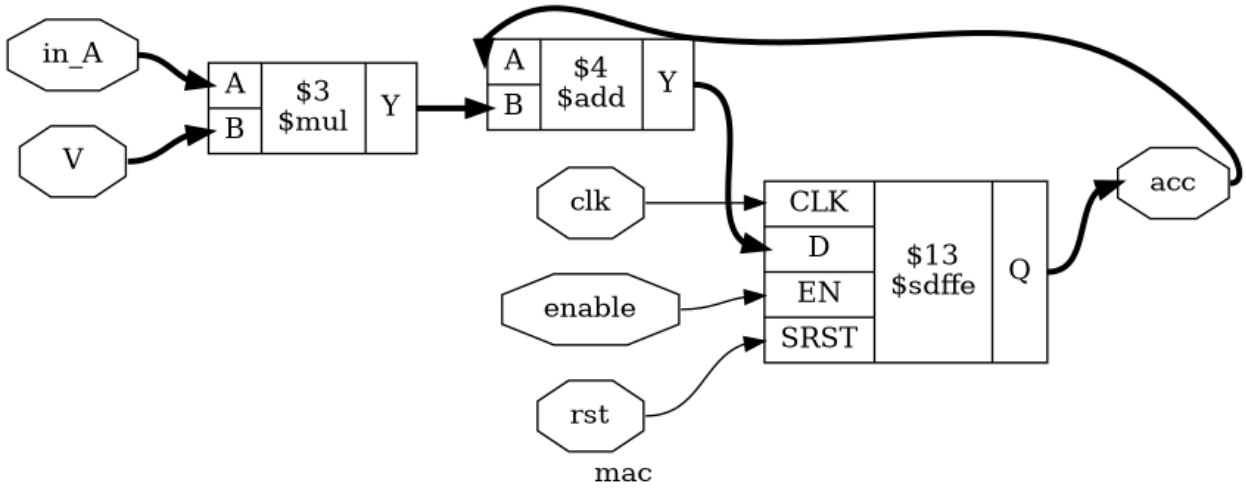
2.3 Terminal based Output reflecting the username through iverilog.

```
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ nano mac.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ ls
locked.v  mac.v  mac.vcd  mac_schematic.dot  mac_schematic.png  mac_sim  sd.v  sd.v
cd  sd_schematic.dot  sd_schematic.png  sd_sim  tb_mac.v  tb_sd.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ iverilog -o mac_sim mac.v tb_ma
c.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ vvp mac_sim
VCD info: dumpfile mac.vcd opened for output.
Time=0 | rst=0 enable=0 | in_A=  0 | acc=  x
Time=5000 | rst=0 enable=0 | in_A=  0 | acc=  0
Time=65000 | rst=1 enable=0 | in_A=  0 | acc=  0
Time=70000 | rst=1 enable=1 | in_A=  2 | acc=  0
Time=75000 | rst=1 enable=1 | in_A=  2 | acc=  8
Time=80000 | rst=1 enable=1 | in_A=  5 | acc=  8
Time=85000 | rst=1 enable=1 | in_A=  5 | acc= 28
Time=90000 | rst=1 enable=0 | in_A= 10 | acc= 28
Time=100000 | rst=1 enable=1 | in_A=  3 | acc= 28
Time=105000 | rst=1 enable=1 | in_A=  3 | acc= 40
Final accumulator = 40
tb_mac.v:45: $finish called at 106000 (1ps)
Time=106000 | rst=1 enable=0 | in_A=  3 | acc= 40
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ gtkwave mac.vcd
```

2.4 Snapshot of waveforms viewed in GTKWave.



2.5 Schematic of the Final Synthesis through YOSYS



3.1 Design of Locked Mac

```
module locked (
   input wire clk,rst,d,
   input wire [7:0]in_A,
   output wire [3:0] out,
   output wire [7:0] acc
);


 wire enable;


 sd u1(clk,rst,d,out);


 assign enable = (out == 4'd4);


 mac u2(clk,rst,enable,in_A,out,acc);


endmodule
```

2.2Testbench Code

```
`timescale 1ns/1ps


module tb_locked;
   parameter R = 65;
   localparam [3:0] V = 4'd4;


   reg clk, rst, d;
   reg [7:0] in_A;
   wire [3:0] out;
   wire [7:0] acc;


   locked uut(clk, rst, d, in_A, out, acc);
```

```
always #5 clk = ~clk;


initial begin
   $dumpfile("locked.vcd");
   $dumpvars(0, tb_locked);
   clk = 0;
   rst = 0;
   d = 0;
   in_A = 0;

   // Apply active-low reset for exactly R ns
   #R;
   rst = 1;
   in_A = 5;

   // Stream sequence 0101 on negative clock edges
   @(negedge clk); d = 0;
   @(negedge clk); d = 1;
   @(negedge clk); d = 0;
   @(negedge clk); d = 1;  @(negedge clk);
   if (out == V)
     $display("FSM output = %0d ,in_A set to R sucessfully!!",out);
   in_A = R;

   @(posedge clk);
   #1;

   $display(" Accumulator = %0d",acc);
  repeat (3) @(negedge clk);              // let the waveform run on

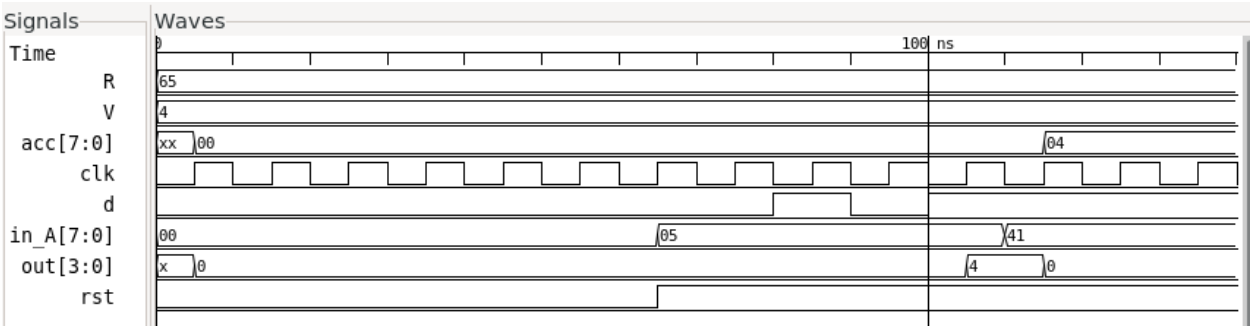
   $finish;
end
```

endmodule

3.3 Terminal based Output reflecting the username through iverilog.

```
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ nano tb_locked.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ iverilog -g2012 -o locked_sim l
ocked.v sd.v mac.v tb_locked.v
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ vvp locked_sim
VCD info: dumpfile locked.vcd opened for output.
FSM output = 4 ,in_A set to R sucessfully!!
 Accumulator = 4
tb_locked.v:48: $finish called at 140000 (1ps)
sachin@Sachin-PC:~/OpenLaneUser/designs/assignment$ gtkwave locked.vcd
```

3.4 Snapshot of waveforms viewed in GTKWave.



3.5 Schematic of the Final Synthesis through YOSYS

